CAMPUS ASSET & SERVICE MANAGEMENT SYSTEM

CASMS

ADVANCED DATABASE MANAGEMENT SYSTEMS
MCA SEMESTER I MINI-PROJECT

Submitted by: [STUDENT NAME]
Register No.: [ROLL NUMBER]
Institution: [COLLEGE / UNIVERSITY]
Department: Master of Computer Applications
Guide: [FACULTY / SUPERVISOR]
Academic Year: [ACADEMIC YEAR]

EDITABLE SUBMISSION DRAFT
Add verified screenshots and actual Phase 6 results before submission.

CONTENTS

- Abstract
- 1. Introduction
- 2. Problem Statement
- 3. Aim and Objectives
- 4. Scope and Limitations
- 5. Stakeholders and Role-Based Operations
- 6. System Architecture
- 7. Database Design
- 8. Technologies and Database Objects
- 9. Implementation by Phase
- 10. Main Business Workflows
- 11. Transactions and Concurrency
- 12. Security Design
- 13. Testing Methodology
- 14. Results and Screenshots
- 15. Achievements, Constraints and Future Work
- 16. Conclusion
- References and Project Artefacts

Abstract

Campus Asset & Service Management System (CASMS) is a local, role-based full-stack web application that maintains an inventory of university equipment and manages its service life. Asset details, departments, rooms, vendors, warranty terms, assignments and transfers are stored in a normalized MySQL database. Staff can submit service tickets, administrators can assign technicians, and technicians can document diagnosis, maintenance work, parts and resolution. The application presents reports, service history, notifications and audit events through a responsive browser interface.

The project demonstrates core Advanced Database Management Systems concepts: relational schema design, DDL and DML, cleaning staged input data, joins, subqueries, aggregation, window functions, views, stored functions, stored procedures, triggers, indexes, transaction management, integrity constraints, and concurrency controls. The backend uses Node.js/Express and parameterized MySQL queries with authenticated sessions and server-side role restrictions. A phased development process separates database evidence from application implementation and formal validation.

Keywords: ADBMS, MySQL, InnoDB, asset management, helpdesk, stored procedures, normalization, transactions, Node.js.

1. Introduction

Colleges depend on desktop computers, projectors, networking devices, lab equipment, furniture and other facilities across multiple departments and rooms. When asset registers and complaint books are maintained separately, it becomes difficult to identify the current location or custodian of equipment, trace service work, monitor pending repairs or estimate maintenance expenditure.

CASMS integrates the equipment register and repair workflow in one relational system. Users access a responsive browser interface, while the server validates requests and calls MySQL for data operations. The implementation is designed for the scale and learning outcomes of an MCA first-semester mini-project, not as a certified production facilities platform.

2. Problem Statement

Manual or isolated spreadsheets make it difficult to keep accurate asset ownership, assignments and transfer history; link service complaints to particular assets; assign work based on technician roles; detect overdue and repeated failures; and maintain an auditable record of maintenance costs and changes. The database must preserve consistency even when a multi-step workflow fails or two operators act concurrently.

3. Aim and Objectives

Aim: Build and demonstrate a browser-based campus asset and service management system whose relational design and operational workflows showcase ADBMS theory and implementation.

Objectives:

1. Model institutional assets, departments, rooms, users, vendors, warranties, requests and maintenance as related MySQL tables.

2. Apply primary keys, foreign keys, uniqueness constraints, checks and normalization rules.
3. Demonstrate data manipulation, cleaning, advanced querying, reusable views and analytical reports.
4. Create stored functions/procedures and triggers for business operations and audit records.
5. Demonstrate COMMIT, ROLLBACK, SAVEPOINT, integrity, row locking and a concurrency lab.
6. Expose scoped workflows through an Express API with authenticated Admin, Staff and Technician roles.
7. Provide usable responsive screens and test them with documented evidence.

4. Scope and Limitations

Included: Master data; accounts and roles; asset registration and history; assignment, transfers and retirement; service request submission and assignment; technician start/hold/resume, repair notes, parts and closure; notifications; dashboard analytics; role-filtered reports with CSV export; audit viewing; documented database performance and transaction exercises.

Deferred/not included in academic release: Attachment uploads, scheduled preventive maintenance automation, external integrations, dedicated mobile apps, cloud production hosting, and enterprise-scale monitoring. Live behavior still requires final verification on the student's installation.

5. Stakeholders and Role-Based Operations

Admin: Configures institutional lookup data and users, maintains inventory, assigns assets and technicians, performs transfers and retirement, reviews comprehensive reports and audit logs.

Staff: Views eligible department/assigned assets, raises service tickets, tracks requests and history, cancels eligible requests and provides feedback.

Technician: Accesses assigned requests, updates repair progress, logs maintenance and parts, closes eligible tickets and uses relevant reports and notifications.

The backend checks identity, role and row scope; displaying a button is not a security decision. Table-level business constraints and stored routines provide an additional consistency layer.

6. System Architecture

Presentation layer: HTML5, responsive CSS and JavaScript pages for authentication, dashboards, inventory, request workflows, management and reporting.

Application layer: Node.js/Express APIs handle input validation, session authentication, role permissions, request states, transactions and standardized error responses.

Database layer: MySQL 8.0/InnoDB with normalized tables, foreign keys, views, indexes, stored routines, audit triggers and transactional integrity.

Logical flow: Browser → Express API → scoped/parameterized query or stored procedure → MySQL → JSON response → UI update. Session records are stored in the additional sessions table.

Development tools: Windows PowerShell, VS Code, MySQL Workbench, Node.js, npm.

Figure placeholders: Insert the verified docs/architecture-diagram.mmd, docs/er-diagram.mmd and docs/application-flow.mmd exports, or corresponding screenshots. The diagrams were drafted during Phase 1 and should be checked against the final schema before submission.

7. Database Design

The implementation has **22 academic domain/support tables** grouped below; the web layer adds **one `sessions` table**, giving an expected **23 base tables** in the completed project.

Campus and identity: roles, departments, buildings, rooms, users, technician_profiles.

Asset register: asset_categories, vendors, asset_statuses, assets, warranties, asset_assignments, asset_transfers, asset_tag_counters.

Service and repair: request_statuses, priorities, service_requests, request_updates, maintenance_records, maintenance_parts.

Operational support: notifications, audit_log.

Web session support: sessions (added in Phase 5).

Relationships include a department to its users/assets, building to its rooms, asset to many service requests and warranty records, service request to maintenance record(s) and timeline updates, and maintenance record to its used parts. An asset can reference a parent asset for components. Current assignments are constrained to avoid multiple active rows for one asset.

Normalization: Roles, statuses, priorities, categories and departments are represented as lookup/reference tables rather than repeated descriptive strings. Many-to-one and one-to-many relations use keys. Assignment, transfer and maintenance parts histories are stored separately to avoid multivalued columns. Actual DDL from database/01_schema.sql is authoritative if early Phase 1 diagrams need reconciliation.

8. Technologies and Database Objects

Area	Implemented technology or artifact
Database engine	MySQL 8.0 / InnoDB
Backend	Node.js, Express, mysql2 raw SQL
Frontend	HTML, CSS, vanilla JavaScript, dashboard charts
Authentication	Server-managed sessions, bcrypt-hashed passwords
Core validation	Foreign keys, checks, unique keys, server input validation
SQL reuse	7 views, 4 stored functions, 6 procedures (Phase 3 target)
Auditing	14 triggers (Phase 3 target)
Query planning	Indexes/EXPLAIN and a separate 50,000-row performance dataset
Deliverables	SQL scripts, APIs/UI, tests, screenshots and documentation

Counts are **targets reported in the Phase 3 batch** and should be verified using database/14_release_integrity.sql rather than assumed in the final submitted report.

9. Implementation by Phase

Phase 1 — Design: Requirements, ER diagram, relational schema, normalization, data dictionary, architecture and application workflow.

Phase 2 — Database construction: Schema DDL, approximately 300 synthetic assets and 600 service requests in the original seed target; cleaning exercises, basic SELECT/WHERE/ORDER BY, aggregations and validation.

Phase 3 — Advanced SQL: Multiple joins, correlated subqueries, GROUP BY/HAVING, ROLLUP, window ranking and running totals, recursive component hierarchy, views, stored routines, triggers, indexing and EXPLAIN.

Phase 4 — Integrity/transactions: Success and failure fixtures for service closure, partial rollback using SAVEPOINT, simulated invalid data, locking/concurrency demonstrations, database accounts and backup/restore.

Phase 5 — Web application: Four development batches: authentication/dashboard/asset browsing; Admin inventory/master-data CRUD; Staff and Technician service workflows; reporting/CSV, notifications, audit and UI polish.

Phase 6 — Testing/documentation: Automated offline checks, non-destructive integration tests, SQL acceptance checks, manual regression matrix, release instructions, screenshots and final submission proof.

10. Main Business Workflows

Asset registration and history: Admin selects department/category/room and submits an asset. A stored procedure registers it and generates a unique tag using counters. Assignment/transfers preserve custody history. Retirement is a controlled status transition rather than deleting the record.

Service request: Staff selects an eligible asset, sets priority and explains the fault. The server creates a ticket and timeline update. Admin assigns an available technician. The technician changes status from Assigned to In Progress, may hold/resume, and records diagnosis, repair work, parts and costs. Closure uses a stored procedure to apply related database changes atomically. Staff can later review closure and, where eligible, give feedback.

Reports/notifications: User-specific notifications and request status updates are accessible through scoped APIs. The reporting module supplies ten defined report types, with Staff/Technician access restricted to four allowed scoped reports and Admin access to all ten.

11. Transactions and Concurrency

The database's transactional features are used to preserve multiple dependent writes. COMMIT confirms a successful operation, ROLLBACK reverses one that fails, and SAVEPOINT demonstrates selective reversal within an open transaction. Locking demonstrations use two independent MySQL Workbench connections to observe conflicts. A successful service closure should save status, maintenance details and parts together; a deliberately invalid closure should not leave partial changes.

Database constraints provide additional protection against orphan references, negative costs, duplicate identifiers and incompatible status transitions. The user performed the Phase 4 batch locally; the final release integrity script and recorded test outcomes provide evidence for submission.

12. Security Design

CASMS uses hashed passwords, a bounded login attempt limiter, regenerated sessions after authentication, httpOnly/sameSite session cookies, protected API routes, scoped SQL predicates, input validation and

parameterized SQL values. Administrative routes check the server-side authenticated role, and changes to notified records are constrained to the signed-in owner.

This is a **localhost academic demo**. The session cookie is configured for HTTP development; it must be hardened for HTTPS deployment. The presence of these protections does not constitute a professional penetration-test certification.

13. Testing Methodology

Three levels are used: (1) offline JavaScript syntax/security-contract tests; (2) live non-destructive HTTP tests for anonymous access, logins, role restrictions, dashboard/assets/requests/reports/CSV, notifications and session invalidation; (3) MySQL integrity and earlier transactional/concurrency labs. A manual checklist covers browser workflow and responsive design. All test results must be recorded as Pass, Fail, Skip or Not Run rather than inferred.

Phase 5 user-reported verification: Batch 1 browser flows passed; Batch 3 service workflows passed; Batch 4 reports/UI passed. Batch 2 features were installed as a prerequisite but not separately confirmed in the conversation. The independent Phase 6 run remains to be documented.

Final verification summary: [INSERT ACTUAL OFFLINE PASS/FAIL COUNTS], [INSERT ACTUAL LIVE PASS/FAIL/SKIP COUNTS], [INSERT SQL VALIDATION], [INSERT BACKUP/RESTORE RESULTS].

14. Results and Screenshots

Include real captures of the following screens with short captions and dates:

1. Login and password change
2. Role-specific dashboard and charts
3. Asset register and new asset form
4. Asset detail, assignment and transfer history
5. Admin master-data and user management
6. Staff new service request
7. Admin ticket assignment
8. Technician repair/status and maintenance closure
9. Closed ticket timeline and feedback
10. Reports filtering/CSV export
11. Notification center
12. Audit viewer
13. Workbench SQL object counts and integrity checks
14. Offline and HTTP test summaries

No screenshots have been inserted automatically because they must represent the app **actually running on the student's computer**.

15. Achievements, Constraints and Future Work

The primary deliverable is a working relational campus operations application with a modern role-based interface, traceable SQL implementation and academically relevant transaction/security experiments. The system is deliberately scoped: no external email system, barcode scanner, mobile native app, attachment store or automatic preventative maintenance scheduler is included in the initial release.

Future enhancement possibilities include QR/barcode scanning, scheduled preventive maintenance, SSO, encrypted cloud backup, file attachments, deployment with HTTPS, structured monitoring and a dedicated mobile client. These are proposed enhancements, not completed features.

16. Conclusion

CASMS demonstrates how database design, advanced SQL and a transaction-aware server can be combined with a responsive interface to manage assets and service work. Its database-first development approach makes normalization, query optimization, integrity, stored routines and access controls directly testable. Final academic sign-off requires the student to attach screenshots and truthful, reproducible Phase 6 results from the local installation.

References and Project Artefacts

1. Official course practical blueprint: ADBMS Practical Blueprint.1docx.docx (source supplied for project planning).

2. Approved project proposal: CASMS_Project_Proposal_v2.docx.

3. CASMS implementation source: database/, server/, public/.

4. Final verification files: tests/, docs/TEST_PLAN_AND_RESULTS.md, database/14_release_integrity.sql.

5. MySQL Reference Manual (InnoDB, transactions, stored programs), Node.js documentation and Express documentation — consult the official documentation relevant to the locally installed versions.

Student declaration: I declare that this report reflects the work completed and verified in my own CASMS installation. [STUDENT SIGNATURE / DATE]